🔹 C / SYSTEMS

Memory Layout of C Program

Q1: What are text, data, bss, heap, stack?
Ans:

- **Text**: executable code (read-only)
- **Data**: initialized global/static variables
- **BSS**: uninitialized global/static variables (zeroed)
- **Heap**: dynamic memory (malloc/free)
- **Stack**: function calls, local variables

Stack vs Heap

Q2: Stack vs Heap?
Ans:

- Stack: automatic, fast, limited, LIFO
- Heap: manual, slower, flexible, large

Pointers & Arrays

Q3: Pointer vs array?
Ans:

- Pointer: variable storing address

- Array: contiguous memory block
- Array name ≠ pointer variable

Q4: Array decay?
Ans:

- In function calls, array decays to pointer to first element

Q5: sizeof(arr) vs sizeof(ptr)?
Ans:

- `sizeof(arr)` → total array size
- `sizeof(ptr)` → size of pointer

Pointer to Array vs Array of Pointers

Q6: `int (*p)[10]` vs `int *p[10]`
Ans:

- `(*p)[10]`: pointer to array of 10 ints
- `p[10]`: array of 10 int pointers

Dynamic Memory

Q7: How malloc works internally?
Ans:

- Requests heap memory
- Uses free lists / bins
- Returns pointer to usable block

Q8: free() does what?
Ans:

- Marks memory reusable
- Does NOT zero memory

Static / Extern / Global

Q9: static keyword meaning?
Ans:

- Limits scope
- Extends lifetime

Q10: extern?
Ans:

- Declares variable defined elsewhere

Function Pointers

Q11: Why function pointers?
Ans:

- Call functions dynamically
- Callbacks, jump tables

Undefined Behavior

Q12: What is UB?
Ans:

- Compiler free to do anything
- Example: out-of-bounds access

🔹 OPERATING SYSTEMS

Process vs Thread

Q1: Process vs thread?
Ans:

- Process: separate address space
- Thread: shared address space

Linux Process Lifecycle

Q2: Process states?
Ans:

- New → Ready → Running → Waiting → Terminated

System Call

Q3: What is system call?
Ans:

- Controlled entry to kernel mode

Kernel Stack

Q4: Kernel stack?
Ans:

- Per-process stack for kernel execution

Context Switch

Q5: What is saved?
Ans:

- Registers, PC, SP, flags

🔷 DATA STRUCTURES & ALGORITHMS

Time Complexity

Q1: Nested loops i=0..n, j=i..n?
Ans: $O(n^2)$

Q2: i*=2 outer, inner n?
Ans: O(n log n)

Q3: outer n, inner j*=2?
Ans: O(n log n)

Distinct Elements (≤2n)

Q4: Optimal approach?
Ans:

- Use counting array

- Time: O(n)
- Space: O(n)

🔷 LINEAR ALGEBRA

Basics

Q1: Vector?
Ans: Ordered list of numbers

Q2: Matrix?
Ans: 2D collection of vectors

Q3: Matrix multiplication meaning?
Ans: Linear transformation

Rank & Determinant

Q4: Rank?
Ans: Number of independent rows

Q5: det = 0 means?
Ans: No inverse, dependent rows

Eigen

Q6: Eigenvector/value?
Ans: Direction unchanged by transform

Q7: Why important?
Ans: PCA, dimensionality reduction

Q8: Eigenvalue = 0?
Ans: Dimension collapse

Inverse

Q9: When inverse exists?
Ans: Square + full rank

Q10: Why pseudo-inverse?
Ans: Non-square / ill-conditioned data

🔹 PROBABILITY & STATISTICS

Foundations

Q1: Conditional probability?
Ans: $P(A|B) = P(A \cap B)/P(B)$

Q2: Bayes theorem meaning?
Ans: Update belief with evidence

Random Variables

Q3: Expectation?
Ans: Long-run average

Q4: Variance?
Ans: Spread of data

Distributions

Q5: Gaussian importance?
Ans: Natural noise model

Q6: Bernoulli?
Ans: Binary outcome

Q7: Poisson?
Ans: Rare events

Theorems

Q8: LLN?
Ans: Average converges to expectation

Q9: CLT?
Ans: Sum → Gaussian

Covariance

Q10: Covariance meaning?
Ans: Joint variation

🔹 MACHINE LEARNING

Basics

Q1: ML definition?
Ans: Learn patterns from data

Q2: Supervised vs Unsupervised?
Ans: Labeled vs unlabeled

Loss & Optimization

Q3: Loss function?
Ans: Measure prediction error

Q4: Gradient descent?
Ans: Iterative error minimization

Q5: Learning rate effect?
Ans: Stability vs speed

Overfitting

Q6: Overfitting?
Ans: Memorizing noise

Q7: Bias-variance tradeoff?
Ans: Simplicity vs flexibility

Linear Models

Q8: Linear regression assumptions?
Ans: Linearity, independence, homoscedasticity

Q9: Feature correlation issue?
Ans: Multicollinearity

Neural Networks

Q10: Vanishing gradient?
Ans: Gradients shrink in deep nets

🔹 ML + SYSTEMS BRIDGE

Q1: Why GPU for ML?
Ans: Massive parallelism

Q2: Why float32?
Ans: Faster, enough precision

Q3: Batch size limited by?
Ans: GPU memory

🔹 RAPID FIRE

1. PCA works because → variance maximization
2. Normalization helps → stable gradients
3. Stack faster → cache + locality
4. Recursion costly → stack frames
5. FP non-associative → rounding
6. ML fails in prod → data drift
7. Large models generalize → implicit regularization